

# Agentic Delegation and the Language Frontier of Software Developers: A Model and Evidence from Claude Code on GitHub

Model, evidence, formalization, and an identification audit

---

**Gabriel Saco**

September 2, 2026

Universidad del Pacífico

Extended presentation of Quispe & Xu (2026) · arXiv:2605.25438v2

[github.com/gsaco/ai-03-quispe/tree/branch1](https://github.com/gsaco/ai-03-quispe/tree/branch1)

# Roadmap and the paper's contribution

## 1. Model

Three production modes generate language-specific entry thresholds and an agent-enabled activation band.

## 2. Evidence

Public GitHub adoption is paired with sharp increases in language breadth, novelty, and entropy.

## 3. Audit

Test whether the threshold is uniquely agentic and whether adoption timing identifies frontier expansion.

## What is new in the paper

It reframes coding AI as a change in the **set of feasible production technologies**, links that mechanism to language portfolios, and builds a developer-month panel around the public Claude coauthor trailer.

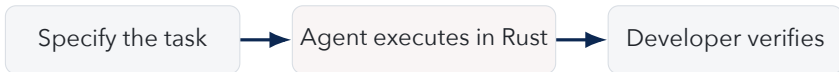
# Production frontier is not a skill frontier

## Skill frontier

Languages the developer can independently understand, write, debug, and maintain. Expansion here is learning or human-capital accumulation.

## Production frontier

Languages in which the developer can ship code using all available tools and collaborators. Expansion here may occur without learning the language.



**The outcome is observable production in a language—not demonstrated mastery of that language.**

# One modelling choice separates frontier expansion from deskilling

## Quispe-Xu: skill is fixed

The agent adds delegated execution to the production menu:

$$\{S, C\} \longrightarrow \{S, C, D\}.$$

Language skill is a state, not a choice. Substitution away from human execution can therefore expand the set of feasible languages without changing the developer's own skill stock.

## Aouad-Lykouris-Zhong: skill is endogenous

AI, effort, and skill are substitutable inputs. More assistance lowers optimal human effort; current effort then determines future skill:

$$a_t \uparrow \Rightarrow e_t^* \downarrow, \quad s_{t+1} = G(s_t, e_t^*).$$

The feedback can shift the steady-state skill distribution downward and generate a long-run productivity paradox.

**Substitutability is common to both papers. The decisive choice is whether reduced human execution feeds back into future skill.**

Aouad, Lykouris & Zhong (2026), *Human-AI Productivity Paradoxes: Modeling the Interplay of Skill, Effort, and AI Assistance*, arXiv:2605.11350.

## Unit of choice

Developer  $i$  receives an opportunity in language  $k$  at time  $t$  with value  $\omega_{ikt}$ . The language is familiar when skill  $s_{ik}$  is high and unfamiliar when it is near a lower bound.

## Two kinds of human capital

Language-specific skill supports direct execution. General ability  $a_i$  supports task decomposition, specification, and verification across languages.

## Technology and costs

Agent capability  $A_t$  raises delegated execution. Each mode also carries execution, interaction, compute, verification, and residual-error costs.

## Observable portfolio

A language is active when the best available mode yields nonnegative certainty-equivalent surplus. Monthly breadth is the number of active languages.

# The developer chooses from a changing menu

## Mode choice

$$m_{ikt}^* \in \arg \max_{m \in \mathcal{M}_g} V_{ikt}^m, \quad Z_{ikt}^g = \mathbb{I}(\max_{m \in \mathcal{M}_g} V_{ikt}^m \geq 0).$$

## Before agent adoption

The menu is  $\mathcal{M}_1 = \{S, C\}$ : solo production or conversational augmentation.

## After agent adoption

The menu becomes  $\mathcal{M}_2 = \{S, C, D\}$  by adding delegated execution.

**Adding a mode cannot shrink the static feasible set; strict expansion requires delegation to dominate at some previously inactive opportunities.**

# Solo production: skill lowers the entry threshold

## Certainty-equivalent surplus

Write solo surplus as opportunity value minus an effective execution cost:

$$V_{ikt}^S = \omega_{ikt} - T_{ik}^S.$$

Greater language-specific skill lowers  $T_{ik}^S$  by making implementation faster and safer.

## Familiar language

A low solo threshold makes a broad set of opportunities profitable even without AI.

## Unfamiliar language

A high solo threshold means only exceptionally valuable projects justify learning and execution costs.

The model treats opportunity value as the source of extensive-margin activation.

# Conversational AI: the no-foothold assumption

## Augmentation gain

Conversational assistance adds a benefit proportional to language skill and subtracts an interaction cost:

$$V_{ikt}^C = V_{ikt}^S + \gamma s_{ik} - r_C.$$

## Assumption 1 for unfamiliar languages

At the lower skill bound, the suggestion gain does not cover interaction cost:

$$\gamma \underline{s} - r_C \leq 0.$$

Therefore conversation does not lower the pre-agent entry threshold for an unfamiliar language.

**This assumption—not the threshold algebra—is what prevents ordinary chat assistance from producing the same frontier expansion.**

# Delegation: execution net of verification and risk

## Delegated surplus

A compact representation is

$$V_{ikt}^D = \omega_{ikt} + \lambda A_t z(a_i) - \kappa(a_i, s_{ik}) - r_D - \rho_{ikt}.$$

## Benefits

Agent capability performs execution; general ability improves specification and task decomposition.

## Costs

Verification, compute, and residual-error risk can erase the execution gain, especially on difficult or high-stakes tasks.

**Delegation expands the frontier only when its net threshold falls below the best pre-agent threshold.**

## Three thresholds summarize the mode comparison



### Pre-agent threshold

Under no foothold, conversation does not beat solo entry for an unfamiliar language, so  $T^1 = T^S$ .

### Delegation threshold

Agent execution matters when  $T^D < T^S$  after every verification, compute, and risk cost is included.

## Proposition 1: menu expansion and weak frontier growth

### Pointwise result

Since the post-agent menu contains every pre-agent option,

$$\max_{m \in \{S, C, D\}} V_{ikt}^m \geq \max_{m \in \{S, C\}} V_{ikt}^m.$$

Thus an opportunity active before adoption remains feasible afterward in the static model.

### From opportunities to language breadth

Summing activation indicators across languages gives weakly greater breadth. Strict growth requires at least one language-opportunity pair for which delegation is profitable while both old modes are not.

**The proposition is a menu-expansion theorem; it does not by itself establish why the new mode is available or whether it is used in equilibrium.**

## Proposition 2: the activation band

### Unfamiliar-language result

If conversation has no foothold and delegation strictly lowers the threshold, then

$$Z_{ikt}^2 - Z_{ikt}^1 = \mathbb{I}(T_{ik}^D \leq \omega_{ikt} < T_{ik}^S).$$

With opportunity distribution  $F$ , the activation probability is

$$F(T_{ik}^S) - F(T_{ik}^D).$$

### All conditions doing work

No conversational foothold; positive net threshold reduction; opportunity mass in the band; sufficient specification and verification ability; and a stable pool of candidate languages.

## Proposition 3: flow, stock, and breadth predictions

### New-language flow

A spike at adoption if the agent immediately activates several previously unused languages.

### Active breadth

A contemporaneous jump that may persist only while projects continue to use the new languages.

### Cumulative stock

A permanent step because every first use remains in the observed historical portfolio.

### Interpretive warning

Cumulative languages mechanically retain adoption-month experimentation. Persistent cumulative growth therefore does not prove continued use, mastery, or a permanent productivity effect.

# Heterogeneity: specialists and general ability

## Specialists

Developers concentrated in few languages have more unfamiliar candidates near the old entry margin. Conditional on comparable opportunities, their first-use response should be larger.

## General ability

Higher ability improves specification and verification, reducing delegated cost and residual risk. The paper proxies ability with prior commit volume.

## Why the empirical mapping is imperfect

Specialization is measured from observed pre-adoption portfolios and ability from activity. Both can also proxy for project type, available time, employer, or GitHub visibility.

## From public trailers to a developer-month panel



### Measurement scope

The pipeline observes public commits and file extensions. It does not observe private repositories, non-committed experimentation, prompt content, who wrote each line, or whether the developer understood the code.

## Final panel

- ▶ 5,346 developers;
- ▶ 2,813 treated early adopters;
- ▶ 2,533 later-adopter controls;
- ▶ 149,688 developer-month observations;
- ▶ January 2024 through April 2026.

## Treatment definition

First public commit with Claude's coauthor trailer. Sustained adopters have at least five Claude commits. Screens remove bots and competing coding-agent signals.

## Control logic

At each cohort and date, not-yet-treated developers supply the counterfactual comparison.

## Four outcome families

### Active languages

Distinct programming languages touched by a developer during the month. Captures contemporaneous breadth.

### Newly used languages

Languages not previously observed for that developer since January 2024. Captures first-use flow.

### Cumulative languages

Total distinct languages ever observed by that month. Captures stock, but mechanically never falls.

### Shannon entropy

Dispersion of changed files across languages. Distinguishes balanced diversification from adding one token language.

**Repository and commit counts are diagnostic outcomes for the scale of activity, not direct measures of the language frontier.**

# Empirical strategy: staggered event study

## Estimator

Doubly robust Callaway-Sant'Anna group-time effects compare each adoption cohort with developers who have not yet adopted. The specification uses a varying base period and one month of anticipation.

## Event window

Six months before through ten months after first public Claude use.

## Inference

1,000 bootstrap iterations with clustering at the developer level.

## Interpretation

The estimator addresses treatment-effect heterogeneity under staggered timing. It does not make voluntary timing exogenous or absorb a project shock that arrives with adoption.

# What the event-study estimand requires

## Maintained conditions

- ▶ conditional parallel trends;
- ▶ no unmodeled anticipation beyond one month;
- ▶ stable treatment definition and no interference;
- ▶ not-yet adopters remain valid controls;
- ▶ outcome histories are measured comparably.

## Most demanding condition

In the absence of Claude adoption, early adopters must not experience a coincident project, employer, or workload shock that independently changes language use.

## Pre-trends

Flat leads are informative about gradual divergence, but have low power and cannot detect a shock that begins exactly at adoption.

## Main estimates: a large adoption-month discontinuity

| Outcome              | Effect at $e = 0$ | Standard error | Scale / interpretation     |
|----------------------|-------------------|----------------|----------------------------|
| Active languages     | 2.528             | 0.063          | pre-adoption mean 0.90     |
| Newly used languages | 1.193             | 0.051          | pre-adoption flow 0.31     |
| Language entropy     | 0.382             | 0.009          | broader file distribution  |
| Cumulative languages | 1.604             | –              | permanent mechanical stock |
| Commits              | 35.08             | –              | strong activity increase   |
| Active repositories  | 1.494             | –              | broader project activity   |

**The magnitudes are economically large and precisely estimated; the central question is what produces the adoption-month discontinuity.**

## Dynamics separate experimentation from persistence

| Outcome              | Adoption month | One month after | Two months after |
|----------------------|----------------|-----------------|------------------|
| Active languages     | 2.528          | 1.227           | 0.693            |
| Newly used languages | 1.193          | 0.126           | approximately 0  |
| Cumulative languages | 1.604          | 1.892           | 2.072            |
| Entropy              | 0.382          | 0.189           | 0.102            |

### What persists

Active breadth and entropy remain positive but decay quickly.

### What does not

The flow of first-use languages returns nearly to zero within two months.

# Outcome-side robustness checks

## Remove first-Claude language

At adoption, active languages still rise by 1.58 and newly used languages by 0.807. The response is not only the language of the first tagged commit.

## Remove all Claude commits

In non-Claude-authored commits, active languages rise by 1.663, newly used by 0.724, and cumulative languages by 1.130 at adoption.

## Additional checks

Baseline activity controls leave results similar; stricter pre-period activity filters reduce magnitudes but preserve significance; results are not driven by one mechanically tagged commit.

**These checks strengthen measurement validity and spillover evidence; they do not independently identify adoption timing.**

# Mechanism diagnostic: breadth works through volume

## Monthly portfolio

Language breadth stays above baseline after adoption because developers make more commits across more repositories and languages.

## Per-commit diversity

The number of languages per commit rises mainly in the adoption month and is not persistently higher afterward.

## Economic reading

Claude adoption is associated with a scale expansion across the developer's monthly portfolio, not a lasting transformation of each individual commit into a more multilingual object.

## Remaining ambiguity

Higher output volume is consistent with lower execution costs, but also with a new deadline, employer, or project that raises both tool demand and coding activity.

# Heterogeneity evidence

## Specialists respond more

Within activity strata, developers with concentrated pre-adoption portfolios show substantially larger first-language use than generalists. This matches the unfamiliar-language margin.

## Illustrative adoption-month effects

High-activity specialists: 0.981 versus 0.301 for generalists. Low-activity specialists: 2.388 versus 1.015.

## What it supports

The pattern is consistent with agents relaxing unfamiliar-language constraints rather than merely intensifying an already broad portfolio.

## What it does not identify

Specialization and activity are endogenous portfolio characteristics. They need not isolate language skill or general ability from project selection.

# What I did: two questions, two counterexamples, one formal audit

## Question 1 · Does the entry-threshold mechanism require an agent? No.

Let any intervention lower the old entry threshold by  $\tau > 0$ . Then

$$T^G = T^1 - \tau, \quad Z^G - Z^1 = \mathbb{I}(T^1 - \tau \leq \omega < T^1).$$

Better libraries, an expert collaborator, or employer support can produce exactly the same activation band.

## Where agent specificity enters

It comes from the mode assumptions: conversation has no foothold in unfamiliar languages, while delegated execution remains valuable after specification, verification, compute, and residual-risk costs. The threshold theorem itself is generic.

## Question 2 · Does the design identify expansion? No.

I construct a project shock that arrives at adoption, directly raises language breadth, and triggers Claude use. It yields flat leads and a positive event-time jump even when Claude's true effect is zero.

**Analytical algebra, a zero-effect simulation, and five Lean proposition targets all point to a narrow verdict: plausible mechanism, incomplete identification.**

## Pressure test II: selection can mimic frontier expansion

### Zero-effect counterexample

Let a project shock  $P_{it}$  trigger adoption and directly raise language breadth:

$$A_{it} = \mathbb{I}(P_{it} \geq c_i), \quad Y_{it} = \mu_i + \lambda_t + \beta P_{it} + \tau A_{it} + \varepsilon_{it}.$$

At adoption, the event-study contrast is

$$\tau + \beta \{\mathbb{E}[P \mid \text{adopt}] - \mathbb{E}[P \mid \text{not yet}]\}.$$

It is positive with  $\tau = 0$  whenever adopters select on larger project shocks.

### Verdict

**Mechanism-consistent, not mechanism-identified.** The evidence documents a large and robust coincidence between Claude adoption and portfolio expansion, but cannot separate delegation from project selection.

### What would be cleaner

Exogenous access or rollout timing; credible instruments; repository-level shocks with unaffected developers; or a comparison against equally large non-agent productivity improvements.

### Repository audit

Lean checks the threshold and selection algebra; with the true Claude effect set to zero, the seeded simulation still estimates a +1.83 adoption-month jump.

# Lean formalization: what is inside the proof boundary

## Result at a glance

Three numbered assumptions and five propositions were inventoried. All five paper-facing proof endpoints compile without `sorry`, `admit`, local axioms, or `native_decide`; the honest status is still **partially formalized** because source fidelity and proof completion are different questions.

## Four-layer architecture

**PaperInterface.lean** · transparent  
source-facing Specs



**ProofInterface.lean** · exact-type  
proof endpoints



**MainTheorems.lean** · lemmas, case  
splits, finite sums



**audit/ + status.json** · source match,  
defects, scope

## Five checked endpoints

1. menu and frontier expansion;
2. unfamiliar-language activation band;
3. corrected cumulative dynamics;
4. specialist and ability heterogeneity;
5. repository expansion.

## Assumptions stay visible

**Assumptions.lean** contains no paper-local postulates. Economic restrictions appear as theorem premises, so Lean verifies implications rather than declaring the assumptions true.

# Lean focus I: the activation band is an exhaustive case split

## 1 • Original mathematical claim

Under the no-foothold condition and  $T^D < T^S$ ,

$$Z^2 - Z^1 = \mathbb{I}\{T^D \leq \omega < T^S\}, \quad \Pr(Z^2 - Z^1 = 1) = F(T^S) - F(T^D) \geq 0.$$

## 2 • Lean statement and decisive proof fragment

```
theorem activationBandIndicator
  (TD TS opportunity : Real)
  (hthreshold : TD < TS) :
  (if TD <= opportunity then 1 else 0) -
  (if TS <= opportunity then 1 else 0) =
  if TD <= opportunity /\ opportunity < TS
  then 1 else 0 := by
  by_cases hD : TD <= opportunity
  . by_cases hS : TS <= opportunity
  . . simp [hD, hS, not_lt_of_ge hS]
  . . simp [hD, hS, lt_of_not_ge hS]
  . have hS : not (TS <= opportunity) := by
    exact fun hs => hD ((le_of_lt hthreshold).trans hs)
```

## 3 • Representation and proof idea

Thresholds and opportunity value are real numbers; activation indicators are zero-one natural numbers. The proof splits on whether delegation and solo production activate. The strict threshold order eliminates the impossible case in which solo activates but delegation does not.

## What Lean verifies

`activationBandForUnfamiliarLanguagesSpec` verifies the indicator identity and nonnegative summed CDF differences. It does not construct the conditional probability measure; continuity is recorded but monotonicity does the order-theoretic work.

# Lean focus: Proposition 3 needs an interior hazard

## 1 • Original mathematical claim and corrected domain

In the closed-frontier benchmark, let

$$g_s = 1 - (1 - p)^{s+1}, \quad g_s < g_{s+1}, \quad (g_{s+2} - g_{s+1}) < (g_{s+1} - g_s).$$

The paper permits  $0 < p \leq 1$  for the strict clause; the strict inequalities are valid only on the interior  $0 < p < 1$ .

## 2 • Lean statement and proof fragment

```
theorem closedFrontierStrictDynamics
  (horizon : Nat) (p : Real)
  (hpPositive : 0 < p) (hpBelowOne : p < 1) :
  let gap := fun h : Nat => 1 - (1-p)^(h+1)
  gap horizon < gap (horizon+1) /\
    gap (horizon+2) - gap (horizon+1) <
    gap (horizon+1) - gap horizon := by
  rw [pow_succ]
  exact mul_lt_of_lt_one_right
    hpowPositive hqBelowOne
```

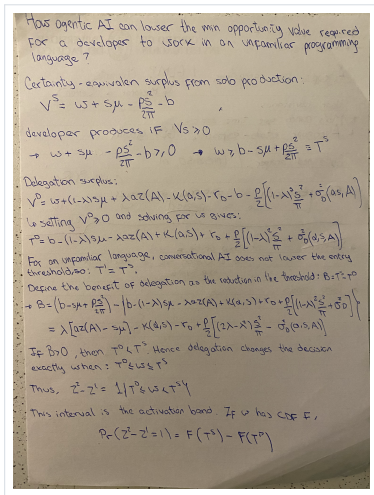
## 3 • Representation and interpretation

Horizon is a natural number; the hazard and gap are real numbers; positivity and the strict upper bound are explicit premises. Rewriting a power as the previous power times  $q = 1 - p$  shows that multiplying a positive survivor term by  $0 < q < 1$  strictly shrinks it.

## What Lean verifies

It verifies the corrected interior theorem exactly, and `closedFrontierHazardOneNotStrict` proves the printed strict claim fails at  $p = 1$ . Full build and the required fast paper check both pass; the overall submission remains honestly partial.

# Where I did not believe the AI: hand derivation and verdict



My derivation: generic threshold equivalence and project-shock selection.

## Threshold verdict

A lower entry threshold does not identify an agent. The agentic interpretation requires the paper's mode-specific assumptions.

## Empirical verdict

A contemporaneous project shock can jointly cause adoption and language expansion. Flat pre-trends and outcome exclusions do not make voluntary adoption timing exogenous.

## Formal verdict

Lean closes all five proposition targets within their encoded scope, but narrows Proposition 3 to  $p < 1$  and does not formalize the event-study identification argument.

**Mechanism-consistent, not mechanism-identified.**